

freedom nonholonomics) and, particularly, as environment dynamics becomes more significant, then the path-planning techniques described above become inadequate for grappling with the full scope of the problem. However, for robots moving in largely flat terrain, the mobility decision-making techniques roboticists use often fall under one of the above categories.

But a path planner can only take into consideration the environment obstacles that are known to the robot in *advance*. During path execution the robot's actual sensor values may disagree with expected values due to map inaccuracy or a dynamic environment. Therefore, it is critical that the robot modify its path in real time based on actual sensor values. This is the competence of *obstacle avoidance* which we discuss below.

6.2.2 Obstacle avoidance

Local obstacle avoidance focuses on changing the robot's trajectory as informed by its sensors during robot motion. The resulting robot motion is both a function of the robot's current or recent sensor readings *and* its goal position and relative location to the goal position. The obstacle avoidance algorithms presented below depend to varying degrees on the existence of a global map and on the robot's precise knowledge of its location relative to the map. Despite their differences, all of the algorithms below can be termed obstacle avoidance algorithms because the robot's local sensor readings play an important role in the robot's future trajectory. We first present the simplest obstacle avoidance systems that are used successfully in mobile robotics. The Bug algorithm represents such a technique in that only the most recent robot sensor values are used, and the robot needs, in addition to current sensor values, only approximate information regarding the direction of the goal. More sophisticated algorithms are presented afterward, taking into account recent sensor history, robot kinematics, and even dynamics.

6.2.2.1 Bug algorithm

The Bug algorithm [101, 102] is perhaps the simplest obstacle avoidance algorithm one could imagine. The basic idea is to follow the contour of each obstacle in the robot's way and thus circumnavigate it.

With Bug1, the robot fully circles the object first, then departs from the point with the shortest distance toward the goal (figure 6.7). This approach is, of course, very inefficient but guarantees that the robot will reach any reachable goal.

With Bug2 the robot begins to follow the object's contour, but departs immediately when it is able to move directly toward the goal. In general this improved Bug algorithm will have significantly shorter total robot travel, as shown in figure 6.8. However, one can still construct situations in which Bug2 is arbitrarily inefficient (i.e., nonoptimal).

A number of variations and extensions of the Bug algorithm exist. We mention one more, the Tangent Bug [82], which adds range sensing and a local environmental represen-

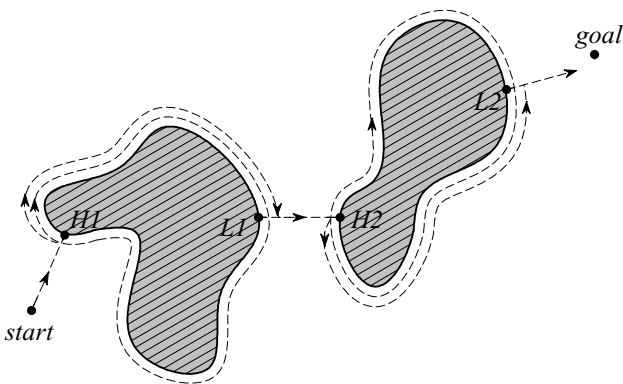


Figure 6.7
Bug1 algorithm with H1, H2, hit points, and L1, L2, leave points [102].

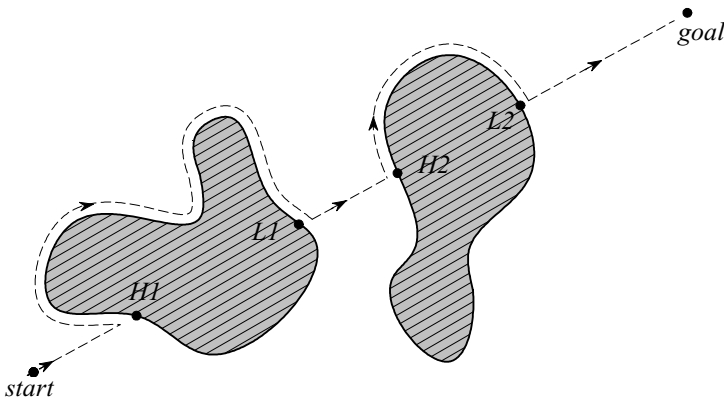


Figure 6.8
Bug2 algorithm with H1, H2, hit points, and L1, L2, leave points [102].

tation termed the local tangent graph (LTG). Not only can the robot move more efficiently toward the goal using the LTG, it can also go along shortcuts when contouring obstacles and switch back to goal seeking earlier. In many simple environments, Tangent Bug approaches globally optimal paths.

Practical application: example of Bug2. Because of the popularity and simplicity of Bug2, we present a specific example of obstacle avoidance using a variation of this technique. Consider the path taken by the robot in figure 6.8. One can characterize the robot's motion in terms of two states, one that involves moving toward the goal and a second that involves moving around the contour of an obstacle. We will call the former state GOAL-SEEK and the latter WALLFOLLOW. If we can describe the motion of the robot as a function of its sensor values and the relative direction to the goal for each of these two states, and if we can describe when the robot should switch between them, then we will have a practical implementation of Bug2. The following pseudocode provides the highest-level control code for such a decomposition:

```
public void bug2(position goalPos){
    boolean atGoal = false;

    while( ! atGoal){
        position robotPos = robot.GetPos(&sonars);
        distance goalDist = getDistance(robotPos, goalPos);
        angle goalAngle = Math.atan2(goalPos, robotPos)-robot.GetAngle();
        velocity forwardVel, rotationVel;

        if(goalDist < atGoalThreshold){
            System.out.println("At Goal!");
            forwardVel = 0;
            rotationVel = 0;
            robot.SetState(DONE);
            atGoal = true;
        }
        else{
            forwardVel = ComputeTranslation(&sonars);
            if(robot.GetState() == GOALSEEK){
                rotationVel = ComputeGoalSeekRot(goalAngle);
                if(ObstaclesInWay(goalAngle, &sonars))
                    robot.SetState(WALLFOLLOW);
            }
            if(robot.GetState() == WALLFOLLOW){
                rotationVel = ComputeRWFrot(&sonars);
                if( ! ObstaclesInWay(goalAngle, &sonars))
                    robot.SetState(GOALSEEK);
            }
        }
        robot.SetVelocity(forwardVel, rotationVel);
    }
}
```

In the ideal case, when encountering an obstacle one would choose between left wall following and right wall following depending on which direction is more promising. In this simple example we have only right wall following, a simplification for didactic purposes that ought not find its way into a real mobile robot program.

Now we consider specifying each remaining function in detail. Consider for our purposes a robot with a ring of sonars placed radially around the robot. This imagined robot will be differential-drive, so that the sonar ring has a clear “front” (aligned with the forward direction of the robot). Furthermore, the robot accepts motion commands of the form shown above, with a rotational velocity parameter and a translational velocity parameter. Mapping these two parameters to individual wheel speeds for each of the two differential drive chassis’ drive wheels is a simple matter.

There is one condition we must define in terms of the robot’s sonar readings, `ObstaclesInWay()`. We define this function to be true whenever any sonar range reading in the direction of the goal (within 45 degrees of the goal direction) is short:

```
private boolean ObstaclesInWay(angle goalAngle, sensorvals sonars) {
    int minSonarValue;
    minSonarValue=MinRange(sonars, goalAngle
                           - (pi/4),goalAngle+(pi/4));
    return (minSonarValue < 200);
} // end ObstaclesInWay() //
```

Note that the function `ComputeTranslation()` computes translational speed whether the robot is wall-following or heading toward the goal. In this simplified example, we define translation speed as being proportional to the largest range readings in the robot’s approximate forward direction:

```
private int ComputeTranslation(sensorvals sonars) {
    int minSonarFront;
    minSonarFront = MinRange(sonars, -pi/4.0, pi/4.0);
    if (minSonarFront < 200) return 0;
    else return (Math.min(500, minSonarFront - 200));
} // end ComputeTranslation() //
```

There is a marked similarity between this approach and the potential field approach described in section 6.2.1.3. Indeed, some mobile robots implement obstacle avoidance by treating the current range readings of the robot as force vectors, simply carrying out vector addition to determine the direction of travel and speed. Alternatively, many will consider short-range readings to be repulsive forces, again engaging in vector addition to determine an overall motion command for the robot.

When faced with range sensor data, a popular way of determining rotation direction and speed is to simply subtract left and right range readings of the robot. The larger the difference, the faster the robot will turn in the direction of the longer range readings. The following two rotation functions could be used for our Bug2 implementation:

```
private int ComputeGoalSeekRot(angle goalAngle) {
    if (Math.abs(goalAngle) < pi/10) return 0;
    else return (goalAngle * 100);
} // end ComputeGoalSeekRot() //

private int ComputeRWFRot(sensorvals sonars) {
    int minLeft, minRight, desiredTurn;
    minRight = MinRange(sonars, -pi/2, 0);
    minLeft = MinRange(sonars, 0, pi/2);
    if (Math.max(minRight,minLeft) < 200) return (400);
                                                // hard left turn
    else {
        desiredTurn = (400 - minRight) * 2;
        desiredTurn = Math.inttorange(-400, desiredTurn, 400);
        return desiredTurn;
    } // end else
} // end ComputeRWFRot() //
```

Note that the rotation function for the case of right wall following combines a general avoidance of obstacles with a bias to turn right when there is open space on the right, thereby staying close to the obstacle's contour. This solution is certainly not the best solution for implementation of Bug2. For example, the wall follower could do a far better job by mapping the contour locally and using a PID control loop to achieve and maintain a specific distance from the contour during the right wall following action.

Although such simple obstacle avoidance algorithms are often used in simple mobile robots, they have numerous shortcomings. For example, the Bug2 approach does not take into account robot kinematics, which can be especially important with nonholonomic robots. Furthermore, since only the most recent sensor values are used, sensor noise can have a serious impact on real-world performance. The following obstacle avoidance techniques are designed to overcome one or more of these limitations.

6.2.2.2 Vector field histogram

Borenstein, together with Koren, developed the vector field histogram (VFH) [43]. Their previous work, which was concentrated on potential fields [92], was abandoned due to the method's instability and inability to pass through narrow passages. Later, Borenstein, together with Ulrich, extended the VFH algorithm to yield VFH+ [150] and VFH*[149].

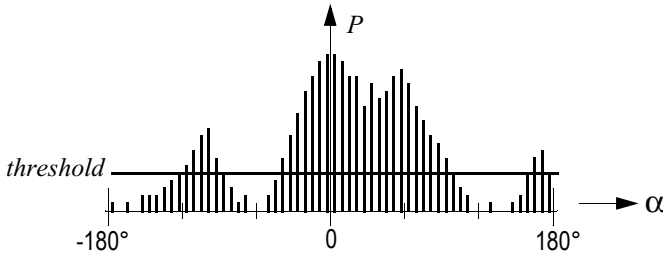


Figure 6.9
Polar histogram [93].

One of the central criticisms of Bug-type algorithms is that the robot's behavior at each instant is generally a function of only its most recent sensor readings. This can lead to undesirable and yet preventable problems in cases where the robot's instantaneous sensor readings do not provide enough information for robust obstacle avoidance. The VFH techniques overcome this limitation by creating a local map of the environment around the robot. This local map is a small occupancy grid, as described in section 5.7 populated only by relatively recent sensor range readings. For obstacle avoidance, VFH generates a polar histogram as shown in figure 6.9. The x-axis represents the angle α at which the obstacle was found and the y-axis represents the probability P that there really is an obstacle in that direction based on the occupancy grid's cell values.

From this histogram a steering direction is calculated. First all openings large enough for the vehicle to pass through are identified. Then a cost function is applied to every such candidate opening. The passage with the lowest cost is chosen. The cost function G has three terms:

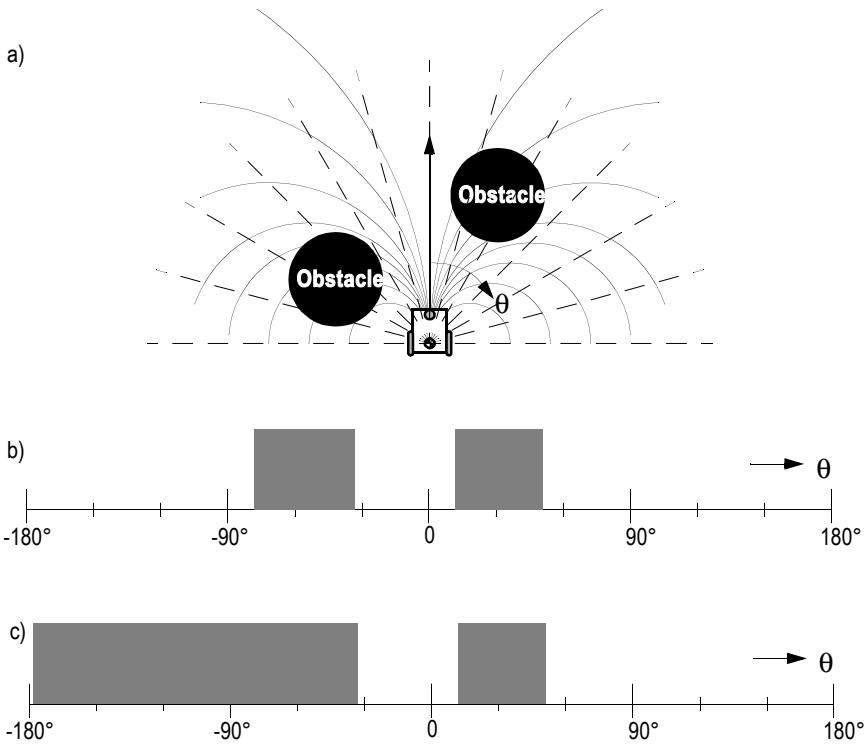
$$G = a \cdot \text{target_direction} + b \cdot \text{wheel_orientation} + c \cdot \text{previous_direction} \quad (6.11)$$

target_direction = alignment of the robot path with the goal;

wheel_orientation = difference between the new direction and the current wheel orientation;

$\text{previous_direction}$ = difference between the previously selected direction and the new direction.

The terms are calculated such that a large deviation from the goal direction leads to a big cost in the term "target direction". The parameters a , b , c in the cost function G tune the behavior of the robot. For instance, a strong goal bias would be expressed with a large value for a . For a complete definition of the cost function, refer to [92].

**Figure 6.10**

Example of blocked directions and resulting polar histograms [54]. (a) Robot and blocking obstacles. (b) Polar histogram. (c) Masked polar histogram.

In the VFH+ improvement one of the reduction stages takes into account a simplified model of the moving robot's possible trajectories based on its kinematic limitations (e.g., turning radius for an Ackerman vehicle). The robot is modeled to move in arcs or straight lines. An obstacle thus blocks all of the robot's allowable trajectories which pass through the obstacle (figure 6.10a). This results in a masked polar histogram where obstacles are enlarged so that all kinematically blocked trajectories are properly taken into account (figure 6.10c).

6.2.2.3 The bubble band technique

This idea is an extension for nonholonomic vehicles of the elastic band concept suggested by Khatib and Quinlan [86]. The original elastic band concept applied only to holonomic

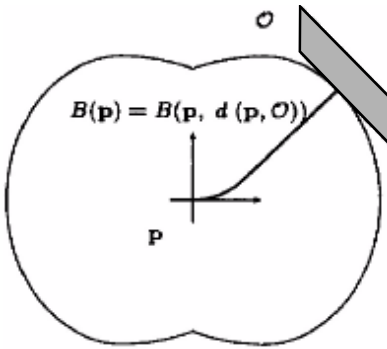


Figure 6.11

Shape of the bubbles around the vehicle (courtesy of Raja Chatila [85]).

vehicles and so we focus only on the bubble band extension made by Khatib, Jaouni, Chatila, and Laumod [85].

A *bubble* is defined as the maximum local subset of the free space around a given configuration of the robot that which can be traveled in any direction without collision. The bubble is generated using a simplified model of the robot in conjunction with range information available in the robot's map. Even with a simplified model of the robot's geometry, it is possible to take into account the actual shape of the robot when calculating the bubble's size (figure 6.11). Given such bubbles, a band or string of bubbles can be used along the trajectory from the robot's initial position to its goal position to show the robot's expected free space throughout its path (see figure 6.12).

Clearly, computing the bubble band requires a global map and a global path planner. Once the path planner's initial trajectory has been computed and the bubble band is calculated, then modification of the planned trajectory ensues. The bubble band takes into account forces from modeled objects and internal forces. These internal forces try to minimize the "slack" (energy) between adjacent bubbles. This process, plus a final smoothing operation, makes the trajectory smooth in the sense that the robot's free space will change as smoothly as possible during path execution.

Of course, so far this is more akin to path optimization than obstacle avoidance. The obstacle avoidance aspect of the bubble band strategy comes into play during robot motion. As the robot encounters unforeseen sensor values, the bubble band model is used to deflect the robot from its originally intended path in a way that minimizes bubble band *tension*.

An advantage of the bubble band technique is that one can account for the actual dimensions of the robot. However, the method is most applicable only when the environment configuration is well-known ahead of time, just as with off-line path-planning techniques.

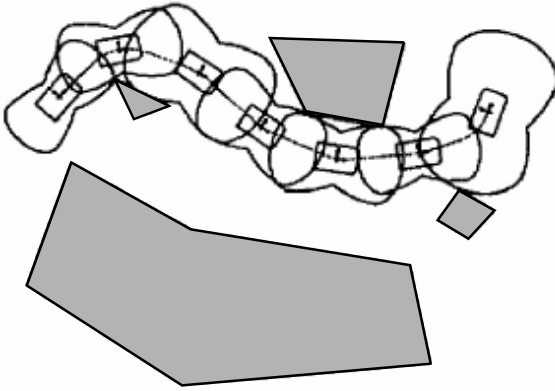


Figure 6.12
A typical bubble band (Courtesy of Raja Chatila [85]).

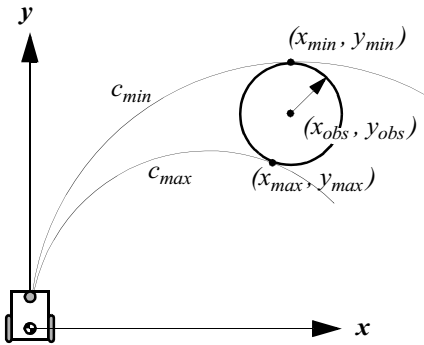
6.2.2.4 Curvature velocity techniques

The basic curvature velocity approach. The curvature velocity approach (CVM) from Simmons [135] enables the actual kinematic constraints and even some dynamic constraints of the robot to be taken into account during obstacle avoidance, which is an advantage over more primitive techniques. CVM begins by adding physical constraints from the robot and the environment to a velocity space. The velocity space consists of rotational velocity ω and translational velocity v , thus assuming that the robot only travels along arcs of circles with curvature $c = \omega/v$.

Two types of constraints are identified: those derived from the robot's limitations in acceleration and speed, typically $-v_{max} < v < v_{max}$, $-\omega_{max} < \omega < \omega_{max}$; and, second, the constraints from obstacles blocking certain v and ω values due to their positions. The obstacles begin as objects in a Cartesian grid but are then transformed to the velocity space by calculating the distance from the robot position to the obstacle following some constant curvature robot trajectory, as shown in figure 6.13. Only the curvatures that lie within c_{min} and c_{max} are considered since that curvature space will contain all legal trajectories.

To achieve real-time performance the obstacles are approximated by circular objects and the contours of the objects are divided into few intervals. The distance from an endpoint of an interval to the robot is calculated and in between the endpoints the distance function is assumed to be constant.

The final decision of a new velocity (v and ω) is made by an objective function. This function is only evaluated on that part of the velocity space that fulfills the kinematic and

**Figure 6.13**

Tangent curvatures for an obstacle (from [135]).

dynamic constraints as well as the constraints due to obstacles. The use of a Cartesian grid for initial obstacle representation enables straightforward sensor fusion if, for instance, a robot is equipped with multiple types of ranging sensors.

CVM takes into consideration the dynamics of the vehicle in useful manner. However a limitation of the method is the circular simplification of obstacle shape. In some environments this is acceptable while, in other environments, such a simplification can cause serious problems. The CVM method can also suffer from local minima since no *a priori* knowledge is used by the system.

The lane curvature method. Ko and Simmons presented an improvement of the CVM which they named the lane curvature method, (LCM) [87] based on their experiences with the shortcomings of CVM. CVM had difficulty guiding the robot through intersections of corridors. The problems stemmed from the approximation that the robot moves only along fixed arcs, whereas in practice the robot can change direction many times before reaching an obstacle.

LCM calculates a set of desired lanes, trading off lane length and lane width to the closest obstacle. The lane with the best properties is chosen using an objective function. The local heading is chosen in such way that the robot will transition to the best lane if it is not in that lane already.

Experimental results have demonstrated better performance as compared to CVM. One caveat is that the parameters in the objective function must be chosen carefully to optimize system behavior.

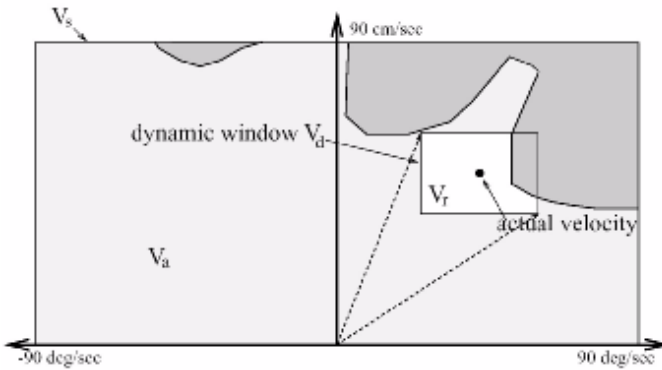


Figure 6.14

The dynamic window approach (courtesy of Dieter Fox [69]). The rectangular window shows the possible speeds (v, ω) and the overlap with obstacles in configuration space.

6.2.2.5 Dynamic window approaches

Another technique for taking into account robot kinematics constraints is the dynamic window obstacle avoidance method. A simple but very effective dynamic model gives this approach its name. Two such approaches are represented in the literature. The dynamic window approach [69] of Fox, Burgard, and Thrun, and the global dynamic window approach [44] of Brock and Khatib.

The local dynamic window approach. In the local dynamic window approach the kinematics of the robot is taken into account by searching a well-chosen velocity space. The velocity space is all possible sets of tuples (v, ω) where v is the velocity and ω is the angular velocity. The approach assumes that robots move only in circular arcs representing each such tuple, at least during one timestamp.

Given the current robot speed, the algorithm first selects a *dynamic window* of all tuples (v, ω) that can be reached within the next sample period, taking into account the acceleration capabilities of the robot and the cycle time. The next step is to reduce the *dynamic window* by keeping only those tuples that ensure that the vehicle can come to a stop before hitting an obstacle. The remaining velocities are called admissible velocities. In figure 6.14, a typical dynamic window is represented. Note that the shape of the dynamic window is rectangular, which follows from the approximation that the dynamic capabilities for translation and rotation are independent.

A new motion direction is chosen by applying an objective function to all the admissible velocity tuples in the dynamic window. The objective function prefers fast forward motion,

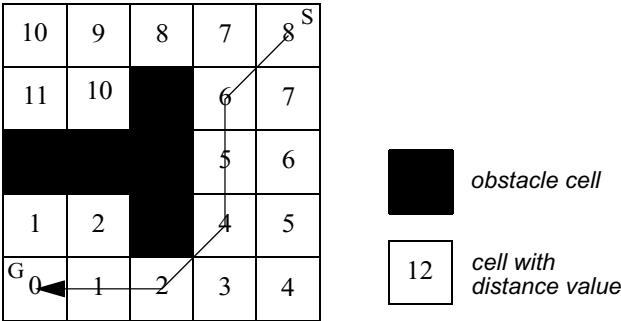


Figure 6.15
An example of the distance transform and the resulting path as it is generated by the NF1 function. S, start; G, goal.

maintenance of large distances to obstacles and alignment to the goal heading. The objective function O has the form

$$O = a \cdot heading(v, \omega) + b \cdot velocity(v, \omega) + c \cdot dist(v, \omega) \tag{6.12}$$

- heading = Measure of progress toward the goal location;
- velocity = Forward velocity of the robot → encouraging fast movements;
- dist = Distance to the closest obstacle in the trajectory.

The global dynamic window approach. The global dynamic window approach adds, as the name suggests, global thinking to the algorithm presented above. This is done by adding *NF1*, or grassfire, to the objective function O presented above (see section 6.2.1.2 and figure 6.15). Recall that NF1 labels the cells in the occupancy grid with the total distance L to the goal. To make this faster the global dynamic window approach calculates the NF1 only on a selected rectangular region which is directed from the robot toward the goal. The width of the region is enlarged and recalculated if the goal cannot be reached within the constraints of this chosen region.

This allows the global dynamic window approach to achieve some of the advantages of global path planning without complete a priori knowledge. The occupancy grid is updated from range measurements as the robot moves in the environment. The NF1 is calculated for every new updated version. If the NF1 cannot be calculated due to the fact that the robot is surrounded by obstacles, the method degrades to the dynamic window approach. This keeps the robot moving so that a possible way out may be found and NF1 can resume.

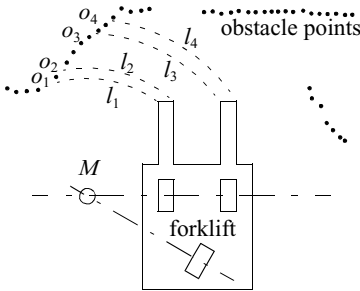


Figure 6.16

Distances l_i resulting from the curvature i_c , when the robot rotates around M (from [128]).

The global dynamic window approach promises real-time, dynamic constraints, global thinking, and minimal free obstacle avoidance at high speed. An implementation has been demonstrated with an omnidirectional robot using a 450 MHz on-board PC. This system produced a cycle frequency of about 15 Hz when the occupancy grid was 30×30 m with a 5 cm resolution. Average robot speed in the tests was greater than 1 m/s.

6.2.2.6 The Schlegel approach to obstacle avoidance

Schlegel [128] presents an approach that considers the dynamics as well as the actual shape of the robot. The approach is adopted for raw laser data measurements and sensor fusion using a Cartesian grid to represent the obstacles in the environment. Real-time performance is achieved by use of precalculated lookup tables.

As with previous methods we have described, the basic assumption is that a robot moves in trajectories built up by circular arcs, defined as curvatures i_c . Given a certain curvature i_c Schlegel calculates the distance l_i to collision between a single obstacle point $[x, y]$ in the Cartesian grid and the robot, depicted in figure 6.16. Since the robot is allowed to be any shape this calculation is time consuming and the result is therefore precalculated and stored in a lookup table.

For example, the search space window V_s is defined for a differential-drive robot to be all the possible speeds of the left and right wheels, v_r, v_l . The dynamic constraints of the robot are taken into account by refining V_s to only those values which are reachable within the next timestep, given the present robot motion. Finally, an objective function chooses the best speed and direction by trading off goal direction, speed, and distance until collision.

During testing Schlegel used a wavefront path planner. Two robot chassis were used, one with synchro-drive kinematics and one with tricycle kinematics. The tricycle-drive